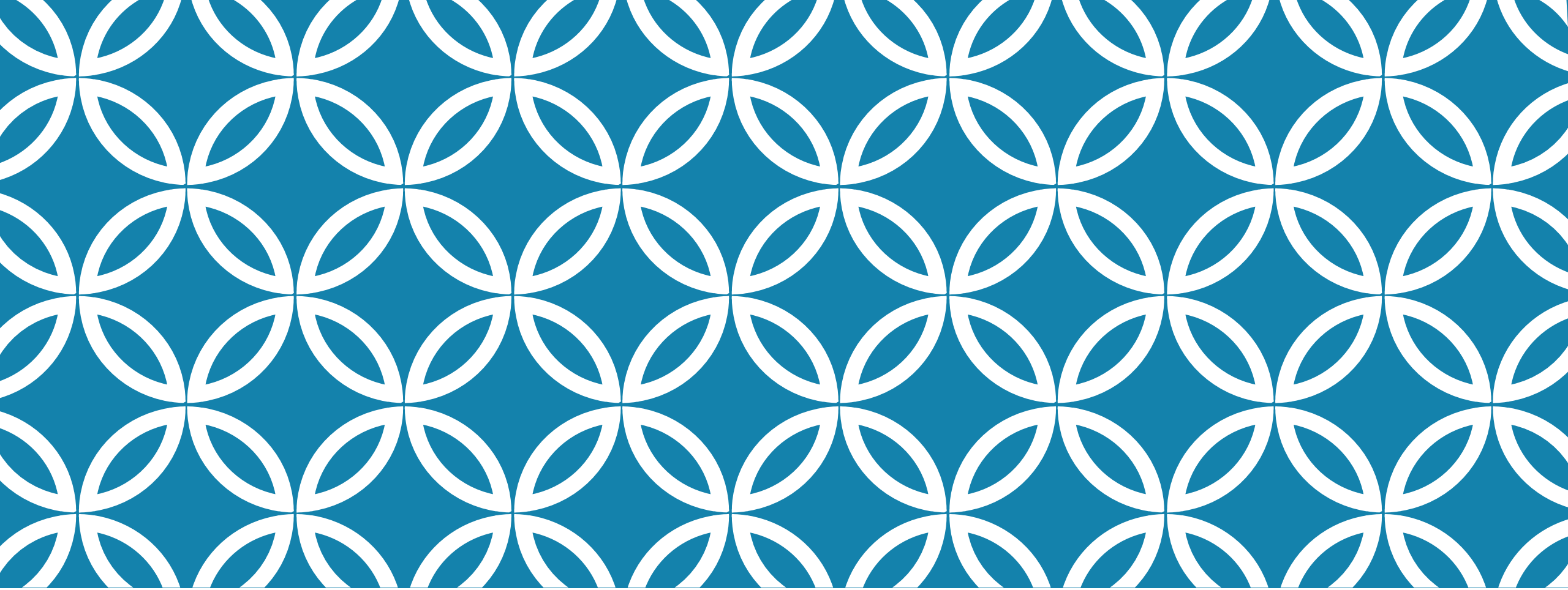


GÖRSEL PROGRAMLAMA-II

Öğr. Gör. Erhan AKAGÜNDÜZ

C# İLE VERİ TABANI İŞLEMLERİ

- **Veri Tabanı Nedir?**
  - Bilgilerin dijital olarak depolandığı alan.
  - Veriler HDD, SSD, NAS veya bulut gibi ortamlarda saklanabilir.
- **Veri Tabanı Seçimi**
  - Yazılım geliştirmede verinin niteliği ve kullanım amacına göre doğru yapı seçimi önemlidir.
  - Küçük projeler: Microsoft Access.
  - Büyük uygulamalar: Microsoft SQL Server, Oracle, MySQL.

C# İLE VERİ TABANI İŞLEMLERİ

- **Örnek Kullanım**
  - **Hastane otomasyon sistemi:** Medikal, finansal ve yönetimsel verilerin yönetimi.
- **Veri Erişim Teknolojileri**
  - Programlama dilleri ile veri tabanı etkileşimi için kullanılır.
  - **Popüler teknolojiler:** ADO.NET, LINQ to SQL, Entity Framework.
- **Bu Derste**
  - ADO.NET'in yapısı, bileşenleri ve çalışma prensibi.
  - C# uygulamaları ile veri tabanı etkileşimleri incelenecek.

VERİ TABANI

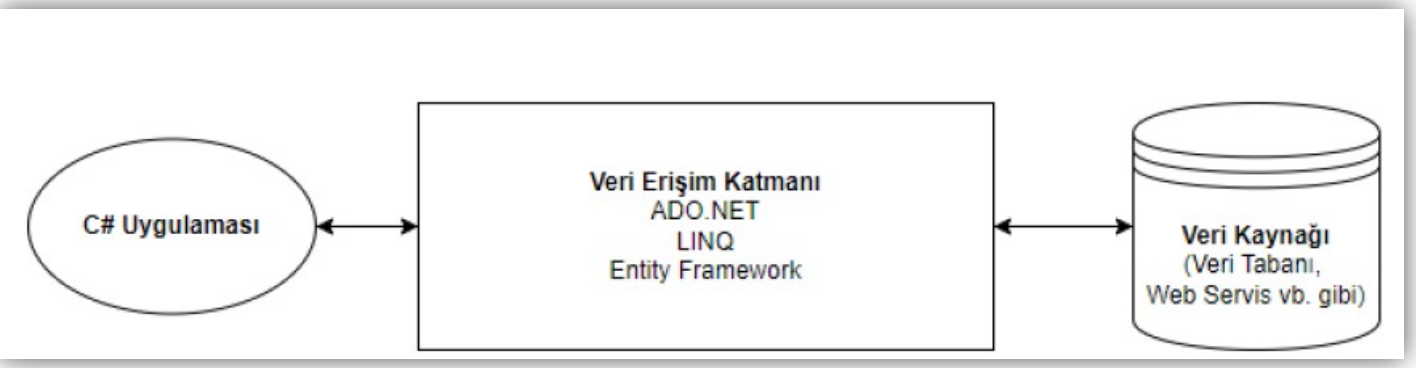
- Verilerin elektronik olarak depolandığı ve sorgulama, ekleme, güncelleme, silme gibi işlemlerin yapılabildiği sistemlerdir.
- Genellikle bir Veri Tabanı Yönetim Sistemi (DBMS) ile kontrol edilir.
- **Veri Tabanı Türleri**

  - İlişkisel Veri Tabanları
  - Nesne Odaklı Veri Tabanları
  - Dağıtılmış Veri Tabanları
  - Veri Ambarları
  - NoSQL Veri Tabanları

  - Grafik Veri Tabanları
  - OLTP Veri Tabanları
  - Bulut Veri Tabanları
  - Çoklu Model Veri Tabanları
  - Belge/JSON Veri Tabanları
  - Kendi Kendini Yöneten Veri Tabanları

İLİŞKİSEL VERİ TABANLARI

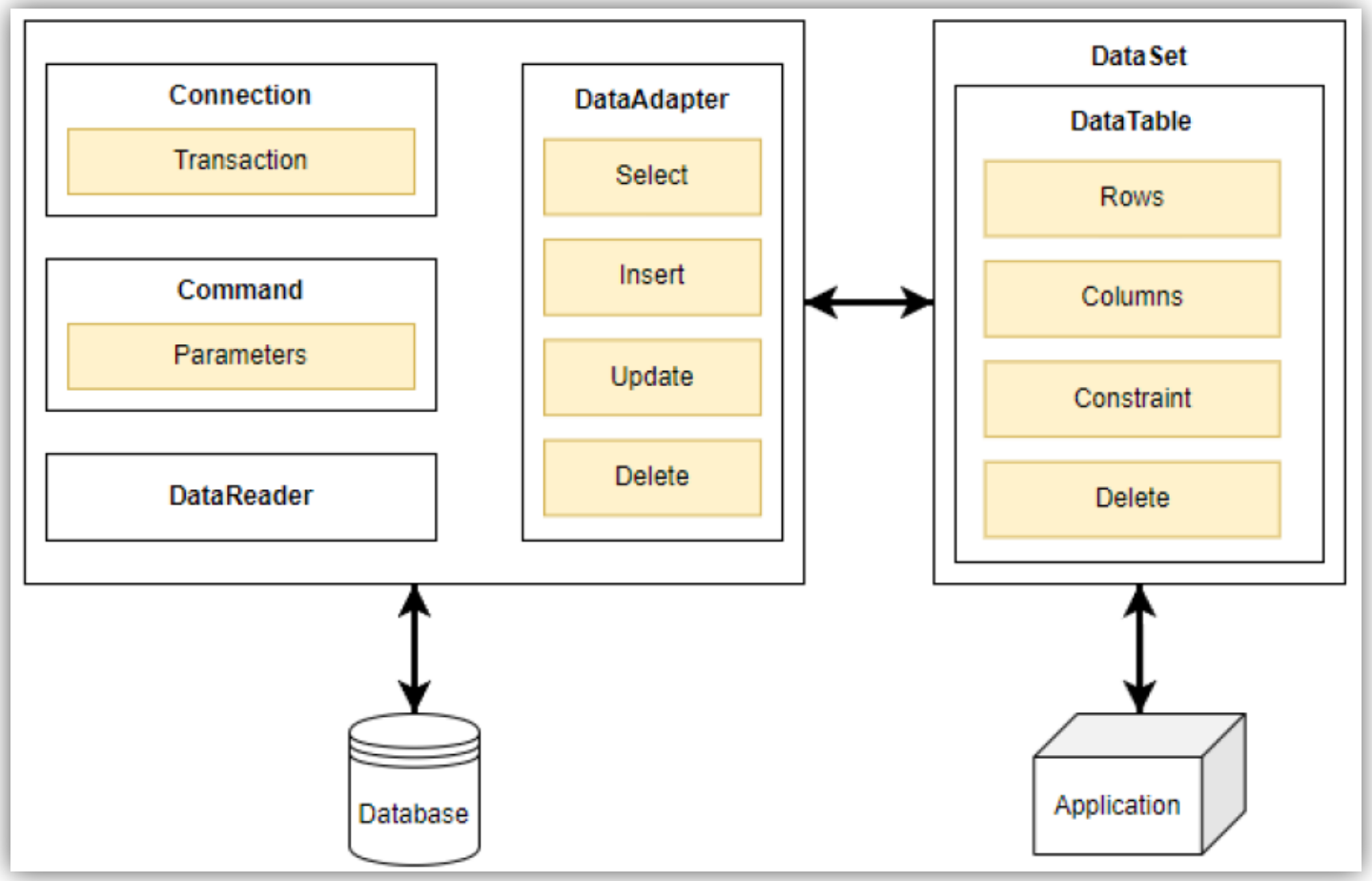
- Birbiriyle ilişkili veri noktalarını depolar.
- Veriler, tablo yapısında satır ve sütunlarda saklanır.
- **Örnekler:** Microsoft SQL Server, Oracle.



ADO.NET (ACTIVEX DATA OBJECT)

- **ADO:** ActiveX Data Object'in kısaltmasıdır. Microsoft tarafından SQL Server, Access, XML gibi veri kaynakları ile işlem yapmak için geliştirilmiştir.
- **ADO.NET:** .NET platformu ile ADO'nun geliştirilmiş halidir. Nesne yönelimli bir yapıya sahiptir.
- **Veri İşlemleri:** ADO.NET, veri tabanı bağlantısı oluşturur ve SQL sorguları çalıştırır. Bağlantı, ihtiyaç durumuna göre açık veya kapalı kalır.
- **Mimari Model:** ADO.NET'in mimari modeli ve bileşenleri, uygulama geliştirme sürecinde önemli bir rol oynar.

ADO.NET MİMARİSİ VE BİLEŞENLERİ



ADO.NET BİLEŞENLERİ

- ADO.NET mimarisinde beş (5) adet bileşen bulunmaktadır.
- Bunlar;
  - Connection,
  - Command,
  - DataReader,
  - DataSet ve
  - DataAdapter'dir.

ADO.NET BİLEŞENLERİ

- **Connection:**
  - Veri kaynağı ile uygulama arasındaki bağlantıyı sağlar.
  - Uygulama ile veri kaynağı arasında bağlantı kurulduğunda Connection nesnesi aracılığı ile SQL komutları çalıştırılır.
  - Connection nesnesi veri kaynağına göre tanımlanmalıdır.
  - SQL Server üzerinde çalışmak için ise SqlConnection sınıfları (class) kullanılmalıdır.

ADO.NET BİLEŞENLERİ

- **Connection:**
  - **SqlConnection** sınıfından türetilerek oluşturulmuş olan **"baglanti"** isimli nesnenin **"ConnectionString"** özelliğine bağlantı dizesi tanımlanmıştır.

```
SqlConnection sqlConnection = new SqlConnection();

sqlConnection.ConnectionString = @"Data Source=localhost;
                                   Database=Hastane;
                                   User Id=sa;
                                   Password=123;";
```

ADO.NET BİLEŞENLERİ

- **Connection:**
  - Bağlantı dizesi içerisinde yer alan parametreler incelenecek olursa;
  - **“Data Source”:** Bağlantı kurulacak olan veri tabanının bulunduğu sunucu ip adresini ya da adını,
  - **“Database”:** Bağlantı kurulacak olan veri tabanı adını,
  - **“User Id”:** MSSQL Server kullanıcı adını,
  - **“Password”:** MSSQL Server kullanıcı şifresini ifade etmektedir.

ADO.NET BİLEŞENLERİ

- Veri kaynağı ile uygulama arasındaki bağlantının durumunu kontrol etmek ve çalışma zamanında program akışını bu doğrultuda yönlendirebilmek için “State” özelliğinden faydalanılır.
- “State” özelliği altı (6) adet değer alabilmektedir.

CONNECTION STATE ÖZELLİĞİNE AİT DURUMLAR

```
    //Bağlantı kapalı.
    Closed = 0,

    //Bağlantı açık.
    Open = 1,

    //Connection nesnesi veri kaynağına bağlanıyor.
    Connecting = 2,

    //Connection nesnesi bir komut çalıştırıyor.
    Executing = 4,

    //Connection nesnesi veri alıyor.
    Fetching = 8,

    //Veri kaynağı ile bağlantının koptuğunu ifade eder.
    //Bu durum, yalnızca bağlantı açıldıktan sonra gerçekleşebilir. Bu du-
rumdaki bir bağlantı kapatılıp yeniden açılabilir.
    Broken = 16
```

ADO.NET BİLEŞENLERİ

- Bağlantı durumunun kontrol edilip eğer bağlantı kapalı ya da kopmuş ise bağlantının tekrar kurulmasını sağlamak için gerekli kodlar aşağıda bulunmaktadır.

```
SqlConnection sqlConnection = new SqlConnection();
sqlConnection.ConnectionString = @"Data Source=localhost;
                        Database=Hastane;
                        User Id=sa;
                        Password=123;";

if (sqlConnection.State == ConnectionState.Closed | sqlConnection.State ==
ConnectionState.Broken)
{
    sqlConnection.Open();
}
```

ADO.NET BİLEŞENLERİ

- **Command:** Command, Connection nesnesinde tanımlanan veri kaynağı üzerinde SQL komutlarının çalıştırılmasını sağlar.
  - **“CommandType”** özelliği ile çalıştırılacak komut tipi belirlenebilir.
  - **“CommandType”** özelliği üç (3) adet değer alabilmektedir.

ADO.NET BİLEŞENLERİ

- SqlCommand sınıfı kullanılarak bir Command nesnesinin tanımlama işlemine yer verilmiştir.

```
//Command nesnesinin tanımlanması.
SqlCommand sqlCommand = new SqlCommand();

//Bağlantı nesnesinin atanması.
sqlCommand.Connection = sqlConnection;

//Komut tipi.
sqlCommand.CommandType = CommandType.Text;

//Çalıştırılmak istenen SQL sorgusu.
sqlCommand.CommandText = "INSERT INTO Klinik (KlinikId,Ad) VALUES (@Kli-
nikId,@Ad)";

//İlgili SQL sorgusuna ait parametreler.
sqlCommand.Parameters.AddWithValue("@KlinikId", Guid.NewGuid());
sqlCommand.Parameters.AddWithValue("@Ad", "Radyoloji");

//Sorgunun çalıştırılması.
sqlCommand.ExecuteNonQuery();

//Veri tabanı bağlantısı kapatılır.
sqlConnection.Close();
```

ADO.NET BİLEŞENLERİ

- **ExecuteScalar():** SQL sorgusu sonucunda geriye bir değer dönüşünün beklendiği durumlarda kullanılır. Örneğin; bir tablodaki yer alan kayıt sayısı.
- **ExecuteNonQuery():** SQL sorgusu sonucunda geriye bir değer dönüşünün beklenmediği durumlarda kullanılır. Örneğin; bir tabloya yapılan kayıt ekleme, silme ve güncelleme işlemleri.
- **ExecuteReader():** SQL sorgusu sonucunda geriye bir liste dönüşünün beklendiği durumlarda kullanılır. Geriye dönen veri, "SqlDataReader" türünden bir nesneye aktarılarak kullanılabilir.
- **ExecuteXmlReader():** ExecuteReader() ile aynı çalışma prensibine sahiptir. Ancak ExecuteReader()'dan farklı olarak XML türünde veri dönüşü için tercih edilir ve veriler; "XmlReader" türünden bir nesneye aktarılarak kullanılabilir.
- **Cancel():** Çalışmakta olan SQL sorgusunu iptal etmek için kullanılır.

DATAREADER

- DataReader bir veri kaynağından veri okumak (readonly) için kullanılır. *Veri tabanından alınan veriler çalışma zamanında Read() metodu ile satır satır okunarak işlenir.*

```
SqlConnection sqlConnection = new SqlConnection();
sqlConnection.ConnectionString = @"Data Source=localhost;
                                   Database=Hastane;
                                   User Id=sa;
                                   Password=123;";
sqlConnection.Open();

SqlCommand sqlCommand = new SqlCommand();
sqlCommand.Connection = sqlConnection;
sqlCommand.CommandText = "Select * from Klinik Order By Ad";

SqlDataReader reader = sqlCommand.ExecuteReader(); //SQL sorgusu çalıştırı-
lıyor.

if (reader.HasRows) //Sorgu sonucunda reader nesnesinde kayıt olup olmadığı
kontrol ediliyor.
{
    while (reader.Read()) //reader içerisindeki satırlar okunmaya başlanı-
yor.
    {
        listBoxKlinikler.Items.Add(reader["Ad"]);//Okunan satırda yer alan
"Ad" sütunundaki değer ListBox'a ekleniyor.
    }
}
```

DATASET

- DataSet içerisinde ilişkisel verileri tablo yapısında barındıran bir DataTable koleksiyonudur.
- Veri tabanından alınan veriler DataSet içerisindeki DataTable'larda saklanır.
- System.Data isim uzayında yer alır ve ADO.NET mimarisinin en önemli bileşenlerinden birisidir.
- DataTable ise verileri satır ve sütunlardan oluşan bir yapı içerisinde bellekte tutarak üzerine işlem yapılmasına olanak sağlayan sınıftır.

DATASET

```
DataSet dataSetHastane = new DataSet(); //DataSet nesnesi tanımlanıyor.

DataTable dataTableKlinik = new DataTable("Klinik"); //Klinik adında bir
tablo nesnesi oluşturuluyor.
dataTableKlinik.Columns.Add("Id", typeof(Guid)); //Oluşturulan tabloya Id
isimli yeni bir sütun ekleniyor.
dataTableKlinik.Columns.Add("Ad", typeof(string)); //Oluşturulan tabloya Ad
isimli yeni bir sütun daha ekleniyor.
dataTableKlinik.PrimaryKey = new DataColumn[] { dataTableKlinik.Co-
lumns["Id"] }; //Id sütunu DataTable için PrimaryKey olarak belirleniyor.

DataRow dr = dataTableKlinik.NewRow(); //Klinik tablosunun şemasını refe-
rans alarak DataRow türünde yeni bir satır oluşturuluyor.
dr["Id"] = Guid.NewGuid(); //Yeni kayıt için gerekli sütun değerleri tanım-
lanıyor.
dr["Ad"] = "Nöroloji";

dataTableKlinik.Rows.Add(dr); //Oluşturulan DataRow tabloya kayıt ediliyor.

dataSetHastane.Tables.Add(dataTableKlinik); //DataTable nesnesi DataSet'e
ekleniyor.
```

DATAADAPTER

- DataAdapter veri kaynağı ile DataSet nesnesi arasında bir köprü olarak tanımlanabilir.
- Diğer bir tanıma göre ise DataAdapter veri kaynağı ile DataSet arasındaki çift taraflı veri akışını kolay, verimli ve performanslı bir şekilde sağlayan sınıftır.
- Aşağıdaki kodda DataAdapter nesnesi ile bir veri tabanından verilerin okunarak bu veriler üzerinde nasıl işlem yapılabileceği gösterilmiştir.
- Yani DataAdapter Nesnesi İle Bir Veri Tabanından Verilerin Okunarak DataSet'e Aktarılması İşlemi gösterilmiştir.

DATAADAPTER

```
string ConnectionString = @"Data Source=localhost;
                                    Database=Hastane;
                                    User Id=sa;
                                    Password=123;";
SqlConnection sqlConnection = new SqlConnection(ConnectionString);

SqlCommand sqlCommand = new SqlCommand();
sqlCommand.Connection = sqlConnection;
sqlCommand.CommandText = "Select * from Klinik Order By Ad";

SqlDataAdapter dataAdapter = new SqlDataAdapter(sqlCommand); //SQL sorgusu
çalıştırılıyor.

//SqlDataAdapter için alternatif kullanım.
//SqlDataAdapter dataAdapter = new SqlDataAdapter("Select * from Klinik Or-
der By Ad",sqlConnection);

DataSet dataSet = new DataSet(); //Bir DataSet nesnesi tanımlanıyor.
dataAdapter.Fill(dataSet); //Sorgu sonucundan elde edilip dataAdapter nes-
nesinde tutulan veriler DataSet'e aktarılıyor.

dataGridView.DataSource = dataSet; //DataSet içerisindeki veriler ekranda
gösterilmek üzere dataGridView'e gönderiliyor.
```